Get Drunk Party Web

Especificación funcional y técnica e instrucciones de implementación para Codex

Versión: 1.0
Fecha: 15 de octubre de 2026
Tipo de proyecto: Aplicación web local de cartas y minijuegos para fiestas
Tecnologías: HTML5, CSS3 y JavaScript puro (ES Modules)
Entorno inicial: PC con Visual Studio Code
Dispositivo objetivo: Teléfono Android
Idioma de la interfaz: Español
Estética: Oscura, urbana, grafiti y neón

---

1. INSTRUCCIÓN PRINCIPAL PARA CODEX

Desarrolla una aplicación web funcional llamada **Get Drunk Party Web**, utilizando HTML, CSS y JavaScript puro, sin frameworks ni dependencias innecesarias.

No construyas únicamente una maqueta visual. Implementa las funciones, las validaciones, la persistencia local, las animaciones, el sistema de turnos, el editor de cartas y los mecanismos de recuperación de errores descritos en este documento.

El proyecto debe ser modular, legible, mantenible y preparado para ampliarse posteriormente.

Reglas de desarrollo

1. No utilices React, Vue, Angular, TypeScript ni frameworks de interfaz.
2. No crees un backend, cuentas de usuario ni servicios en la nube.
3. No dependas de una API externa para que funcione el juego.
4. No utilices imágenes como requisito para las cartas. La interfaz se construirá con HTML, CSS, tipografía, iconos y efectos visuales.
5. Utiliza JavaScript ES Modules y una arquitectura de archivos clara.
6. Implementa un catálogo inicial de 50 cartas originales: diez por cada una de las cinco categorías.
7. No hardcodees las cartas dentro de las funciones de juego. El contenido debe estar separado de la lógica.

8. Todas las cartas deben poder editarse, desactivarse, duplicarse y eliminarse desde un editor integrado.
9. El dado tiene seis caras equiprobables, pero solo cinco corresponden a categorías de cartas.
10. La sexta cara, SHOT, es una acción directa del dado. No debe existir un mazo de cartas SHOT.
11. El proyecto debe poder ejecutarse inicialmente en el PC y abrirse desde un teléfono Android conectado a la misma red Wi-Fi.
12. Diseña la arquitectura para que posteriormente pueda ejecutarse en el propio Android mediante un servidor HTTP local, sin depender del PC.
13. No inventes funciones que contradigan estas reglas. Si falta una decisión indispensable, utiliza la alternativa más sencilla y documenta el supuesto.

Primero inspecciona el proyecto existente, si lo hay. Después implementa la aplicación completa, comprueba las rutas, ejecuta las pruebas disponibles y corrige los errores encontrados. No te limites a explicar cómo habría que programarla.

---

2. OBJETIVO DEL PROYECTO

Crear una aplicación web para jugar con un grupo de personas mediante un dado virtual, cartas aleatorias, retos, preguntas, premios, castigos y minijuegos.

El juego toma inspiración de las mecánicas generales de juegos de fiesta, pero utiliza un catálogo propio de cartas, textos, instrucciones, iconos y diseño visual.

La aplicación se manejará desde un único dispositivo, normalmente un teléfono colocado en el centro del grupo.

Funciones principales

- Registrar y administrar participantes.
- Mantener un orden fijo de turnos.
- Lanzar un dado virtual de seis caras.
- Seleccionar una carta aleatoria de la categoría obtenida.
- Evitar repeticiones hasta agotar el mazo de esa categoría.
- Evitar que la primera carta de un nuevo ciclo sea la última del ciclo anterior, siempre que haya alternativas.
- Mostrar una animación de volteo de la carta.
- Asignar participantes según las instrucciones de cada carta.
- Mostrar el nivel de picante mediante iconos de fuego.
- Configurar unidades de penalización y tipos de bebida.
- Registrar si una carta se completó, se pasó o se eligió una alternativa.
- Consultar el manual de un minijuego mediante un botón flotante de ayuda.
- Crear, editar, duplicar, desactivar y eliminar cartas desde la aplicación.

- Guardar los cambios en el navegador.
- Importar y exportar cartas y configuraciones mediante JSON.
- Funcionar correctamente en teléfonos Android y en navegadores de escritorio.

Funciones fuera del alcance inicial

No implementar:

- Multijugador por internet.
- Inicio de sesión.
- Base de datos remota.
- Chat.
- Puntuaciones.
- Temporizadores obligatorios.
- Automatización de las reglas físicas de los minijuegos.
- Registro de alcohol en sangre.
- Integraciones con redes sociales.
- Dependencia de imágenes externas.

Los minijuegos se presentan con sus reglas. El grupo los realiza físicamente y después indica que el turno terminó.

---

3. CATEGORÍAS Y DADO

Las cinco categorías de cartas son:

ID	Categoría	Color
`VERDAD`	Verdad	Verde neón `#39E58C`
`DESAFIO`	Desafío	Magenta `#FF3CAC`
`MINIJUEGO`	Minijuego	Azul eléctrico `#36C5FF`
`PREMIO`	Premio	Amarillo `#FFD447`
`CASTIGO`	Castigo	Rojo `#FF4D5A`

La sexta cara del dado es:

ID	Acción	Color

`SHOT`	Acción directa de tomar un shot según las reglas de la partida	Gris claro `#D5D5DF`

`SHOT` no es una categoría de cartas.

Probabilidades

El dado debe seleccionar una de sus seis caras con la misma probabilidad teórica:

- VERDAD: 1/6.
- DESAFIO: 1/6.
- MINIJUEGO: 1/6.
- PREMIO: 1/6.
- CASTIGO: 1/6.
- SHOT: 1/6.

Implementa el resultado mediante JavaScript y después representa ese resultado con la animación. La animación no debe decidir el resultado.

Si sale SHOT:

1. Mostrar una animación breve del dado.
2. Mostrar un aviso claro: «SHOT: esta cara no tiene carta».
3. Mostrar el botón «Registrar resultado».
4. Permitir registrar que se realizó la acción o que se eligió una alternativa sin alcohol.
5. Finalizar el turno y pasar al siguiente participante.

No extraer una carta, no acceder a un mazo de shots y no cambiar la categoría obtenida.

No volver a lanzar el dado automáticamente cuando una categoría esté agotada. Mostrar una ventana que permita reiniciar ese mazo o cancelar la tirada. La cancelación no debe cambiar silenciosamente las probabilidades.

La aplicación debe permitir configurar las alternativas sin alcohol y evitar que un resultado del juego obligue a beber.

---

4. IDENTIDAD VISUAL

Estética general

Diseña una interfaz oscura, urbana y grafitera, con una apariencia de juego de fiesta moderno.

Utiliza:

- Fondo negro carbón: `#0B0B0F`.
- Superficies: `#15151C`.
- Bordes: `#30303A`.
- Texto principal: `#F5F5F7`.
- Texto secundario: `#A6A6B3`.
- Colores de categoría definidos anteriormente.
- Acentos neón moderados.
- Bordes redondeados.
- Sombras y brillos sutiles.
- Texturas CSS opcionales que no dificulten la lectura.
- Tipografía sans-serif legible.
- Estilo de grafiti únicamente como acento visual.

Evita saturar la interfaz con animaciones, destellos o efectos que dificulten la lectura.

El diseño debe ser original. No reproduzcas logotipos ni copies literalmente el diseño de las cartas de otros juegos.

Reverso de las cartas

Cada categoría tendrá un reverso reconocible por su color y por un diseño geométrico inspirado en cintas cruzadas.

Debe incluir:

- El nombre de la categoría repetido sobre las cintas.
- Una composición en forma de X.
- El color identificador de la categoría.
- Un estilo oscuro y urbano.

No es necesario incluir un logotipo definitivo.

Frente de las cartas

El frente debe ser simple y contener:

1. Participante o participantes en la zona superior centrada, cuando corresponda.
2. Pregunta, reto, premio, castigo o nombre del minijuego en el centro.
3. Unidades de penalización en la esquina inferior izquierda, si corresponden.
4. Iconos de fuego en la esquina inferior derecha.
5. Botón flotante de ayuda fuera de la superficie de la carta.

El texto principal debe tener suficiente espacio y ajustarse automáticamente a la longitud de la descripción.

Animación de volteo

Implementa una animación CSS 3D con `perspective` y `backface-visibility`.

Flujo:

1. Mostrar el reverso de la carta.
2. Animar el giro.
3. Revelar el frente.
4. Mostrar el texto y los participantes asignados.
5. Permitir resolver el turno.

La carta seleccionada y su asignación deben guardarse antes de comenzar la animación. No debe seleccionarse una carta nueva porque la animación termine o falle.

Respeta `prefers-reduced-motion` y proporciona una alternativa sin animaciones intensas.

---

5. RESPONSIVE DESIGN

La prioridad absoluta es el teléfono Android.

Implementa un diseño mobile-first.

En teléfonos

- Una columna principal.
- Dado visible y accesible.
- Botón de lanzamiento grande.
- Carta centrada y legible.
- Botones de resolución cómodos para los dedos.
- Navegación sencilla entre partida, participantes, editor y manuales.
- Sin desplazamiento horizontal.
- Modales adaptados a pantallas pequeñas.
- El botón de ayuda debe quedar fuera de la carta y no tapar su texto.
- Los controles deben respetar las áreas seguras de la pantalla.

En tabletas y ordenadores

- Aprovechar el espacio adicional.
- Mantener el dado y la carta como elementos principales.
- Mostrar listas de cartas y participantes con más información cuando haya espacio.

Utiliza CSS Grid, Flexbox y unidades relativas. No dependas de dimensiones fijas que rompan el diseño en dispositivos pequeños.

---

6. PARTICIPANTES Y TURNOS

El juego debe comenzar con una pantalla de configuración de participantes.

Participantes iniciales de prueba

- Sharon.
- Camilo.
- Valentina.
- Andrés.

Estos nombres son ejemplos editables.

Funciones del módulo

- Agregar un participante.
- Editar su nombre.
- Eliminarlo.
- Mostrar todos los participantes activos.
- Mantener el orden de registro.
- Mostrar quién tiene el turno actual.
- Avanzar al siguiente participante al terminar un turno.
- Recuperar el orden al recargar la página.
- Evitar nombres vacíos.
- Permitir nombres repetidos si tienen IDs distintos, aunque se recomienda avisar al usuario.

Cada participante tendrá un ID estable, un nombre y preferencias opcionales de elegibilidad para dinámicas sensibles.

Ejemplo:

```
{
  id: "player_001",
  name: "Sharon",
  active: true,
  preferences: {
    avoidPairKissWithMen: false
  }
}
```

No deduzcas el género ni la orientación sexual a partir del nombre. Si se habilita una restricción de compatibilidad, debe configurarse explícitamente por el usuario.

Orden de turnos

El orden es secuencial y circular.

Ejemplo:

Sharon → Camilo → Valentina → Andrés → Sharon.

El índice del turno debe actualizarse de manera segura cuando se elimine al participante actual o cuando se agreguen participantes durante una partida.

Si una carta requiere dos jugadores, la asignación no debe modificar el orden de turnos. El siguiente turno corresponde al siguiente participante registrado.

Posiciones alrededor de la mesa

El orden de registro representará el orden circular de los jugadores sentados.

- `LEFT_PLAYER`: siguiente participante en la lista circular.
- `RIGHT_PLAYER`: participante anterior en la lista circular.

La orientación física de izquierda y derecha debe ser configurable porque depende de cómo estén sentadas las personas.

Para cartas que indiquen frente, diagonal o una posición no deducible de la lista, permite seleccionar manualmente al participante objetivo. No inventes una geometría de mesa que la aplicación desconoce.

---

7. REGLAS DE ASIGNACIÓN Y CONSENTIMIENTO

Cada carta podrá indicar un modo de asignación.

Modo	Comportamiento
`CURRENT_PLAYER`	Jugador actual.
`RANDOM_PLAYER`	Jugador aleatorio elegible.

`NEXT_PLAYER`	Siguiente participante.
`PREVIOUS_PLAYER`	Participante anterior.
`LEFT_PLAYER`	Persona a la izquierda según el orden circular.
`RIGHT_PLAYER`	Persona a la derecha según el orden circular.
`PLAYER_PAIR`	Dos participantes elegibles distintos.
`ALL_PLAYERS`	Grupo completo.
`MANUAL_SELECTION`	El usuario selecciona a los participantes.

Si no existe una asignación válida, mostrar un aviso y permitir seleccionar otra carta o pasar el turno.

Restricción opcional de besos

Incluye un botón con un icono de arcoíris en la pantalla de configuración de participantes o de partida.

Nombre de la función: «Restricción de besos».

Cuando esté activada:

- Evitar asignar un beso de pareja entre dos hombres cuando esa sea la restricción configurada por el grupo.
- Permitir dinámicas grupales de tres o más participantes únicamente si todos aceptan participar.
- No asumir que una dinámica grupal es automáticamente aceptable para todos.
- No cambiar silenciosamente una carta que indique una posición concreta.
- Si no hay una combinación compatible, ofrecer una alternativa o permitir pasar la carta.

La preferencia debe ser configurable y guardarse localmente. No infieras identidades ni orientación sexual.

Cualquier dinámica de contacto físico debe poder rechazarse. No conviertas el rechazo en una obligación de beber.

---

8. MODELO DE DATOS DE LAS CARTAS

Cada carta será un objeto JavaScript independiente.

Ejemplo:

```
{
  id: "DESAFIO_001",
  categoryId: "DESAFIO",
  title: "Reto de grupo",
  description: "Propón una imitación para el grupo.",
  icon: {
    type: "class",
    value: "fa-solid fa-fire"
  },
  spiceLevel: 2,
  target: {
    mode: "CURRENT_PLAYER",
    count: 1,
    position: null
  },
  penalty: {
    enabled: true,
    units: 1,
    drinkTypeId: "NORMAL"
  },
  manualId: null,
  enabled: true
}
```

Campos

- `id`: identificador único e inmutable.
- `categoryId`: categoría de la carta.
- `title`: título corto.
- `description`: texto principal.
- `icon.type`: clase CSS, URL autorizada o SVG validado.
- `icon.value`: valor del icono.
- `spiceLevel`: entero entre 0 y 5.
- `target.mode`: modo de asignación.
- `target.count`: cantidad de participantes requerida.
- `target.position`: posición relativa, si aplica.
- `penalty.enabled`: indica si hay penalización configurada.
- `penalty.units`: número de unidades configuradas.
- `penalty.drinkTypeId`: referencia al catálogo de bebidas.

- `manualId`: referencia al manual de un minijuego.
- `enabled`: permite desactivar una carta.

Para cartas que no tengan penalización, utiliza `enabled: false` dentro del objeto `penalty`, o define un valor nulo de forma consistente. No interpretes la ausencia de un dato como una penalización automática.

Reglas de validación

- IDs únicos.
- Categorías existentes.
- Descripciones no vacías.
- Nivel de picante entre 0 y 5.
- Unidades de penalización no negativas.
- Modo de asignación reconocido.
- Participantes requeridos compatibles con el número de jugadores.
- Manual existente cuando `manualId` no sea nulo.
- Iconos validados según su tipo.

No utilices `eval`, `innerHTML` con contenido no confiable ni ejecutes código aportado por el usuario desde el editor.

---

9. SISTEMA DE ICONOS

El editor debe permitir cambiar el icono de una carta sin tocar el código de la lógica del juego.

Tipos admitidos:

1. Clase CSS de una biblioteca local incluida en el proyecto.
2. URL de un icono SVG o PNG autorizada.
3. SVG propio validado.

El editor debe incluir:

- Selector del tipo de icono.
- Campo para el valor del icono.
- Vista previa.
- Tamaño visual uniforme.
- Posición automática.
- Validación del dato.

Para evitar dependencia de internet, utiliza iconos SVG propios o una biblioteca de iconos incluida localmente. No dependas de un CDN para los iconos esenciales.

No aceptes bloques de HTML o JavaScript arbitrario como iconos.

---

10. SISTEMA DE FUEGOS

El nivel de picante se representa mediante entre cero y cinco iconos de fuego.

Nivel	Representación
0	Sin fuegos.
1	Un fuego.
2	Dos fuegos.
3	Tres fuegos.
4	Cuatro fuegos.
5	Cinco fuegos.

Los fuegos se muestran en la esquina inferior derecha de la carta.

El editor permite cambiar el nivel de picante mediante un selector de 0 a 5.

El nivel de picante es una clasificación temática del reto, no una medida de alcohol ni una indicación de que una persona deba realizar una actividad.

---

11. CATÁLOGO DE BEBIDAS Y PENALIZACIONES

El catálogo de bebidas será independiente del catálogo de cartas.

Ejemplo:

```
const drinkTypes = [
  {
    id: "NORMAL",
    name: "Shot estándar",
    unitLabel: "unidad",
    enabled: true
  },
```

```
  {
    id: "ALTERNATIVE",
    name: "Alternativa sin alcohol",
    unitLabel: "opción",
    enabled: true
  }
];
```

El catálogo se podrá editar desde la aplicación.

Funciones:

- Agregar un tipo de bebida.
- Cambiar su nombre.
- Activarlo o desactivarlo.
- Eliminarlo si ninguna carta lo utiliza.
- Mostrar las unidades de penalización configuradas en cada carta.

Las cartas podrán indicar una cantidad y un tipo de unidad. El editor debe permitir cambiar esas reglas sin modificar la lógica general.

No equipares automáticamente bebidas de distinta graduación o volumen. El sistema no debe presentar las unidades configuradas como equivalentes exactos de alcohol.

Si una persona no quiere realizar un reto o beber, debe poder pasar la actividad o escoger una alternativa sin alcohol. La aplicación no debe imponer consumo como condición obligatoria.

Para la cara SHOT del dado, utiliza una regla de acción directa y permite configurar una alternativa sin alcohol.

---

12. MAZOS Y PREVENCIÓN DE REPETICIONES

Cada categoría tendrá su propio mazo.

Reglas obligatorias:

1. Obtener las cartas activas de una categoría.
2. Barajar aleatoriamente el mazo.
3. Seleccionar la siguiente carta del mazo.
4. Marcarla como utilizada.
5. No volver a seleccionarla durante el mismo ciclo.
6. Al agotar el mazo, crear un nuevo orden aleatorio.

7. Impedir que la primera carta del nuevo ciclo sea la última carta del ciclo anterior, si existe al menos otra carta elegible.
8. Mantener mazos independientes por categoría.
9. Guardar el progreso de los mazos en el estado de la partida.
10. Permitir reiniciar un mazo agotado.

Si una categoría contiene una sola carta activa, se permite repetirla porque no existe otra alternativa.

Si se edita o elimina una carta durante una partida, actualizar el mazo sin corromper el estado. Una carta ya utilizada no debe volver a aparecer por el simple hecho de editar otra carta.

No uses selección aleatoria con reemplazo para cada tirada. Implementa un mazo barajado y un índice de lectura.

---

13. FLUJO DE TURNO

El flujo de cada turno será:

1. Mostrar al jugador actual.
2. Habilitar el botón «Lanzar dado».
3. Al pulsarlo, bloquear inmediatamente nuevos lanzamientos.
4. Obtener el resultado aleatorio.
5. Ejecutar la animación del dado.
6. Si el resultado es SHOT, mostrar la acción directa y no extraer ninguna carta.
7. Si el resultado es una categoría, obtener la siguiente carta válida.
8. Asignar los participantes según las reglas de la carta.
9. Ejecutar la animación de volteo.
10. Mostrar el frente de la carta.
11. Permitir consultar el manual si existe.
12. Permitir registrar el resultado del turno.
13. Guardar el resultado.
14. Avanzar al siguiente participante.
15. Desbloquear el dado.

Estados mínimos del juego:

- `READY`
- `ROLLING`
- `DRAWING_CARD`
- `REVEALING`
- `RESOLVING`
- `SAVING`

- `ERROR`

Controla los estados mediante una máquina de estados sencilla o una estructura equivalente. Evita que dos operaciones modifiquen el turno simultáneamente.

Los resultados de una carta pueden ser:

- `COMPLETED`
- `PASSED`
- `ALTERNATIVE_SELECTED`

El resultado debe registrarse sin imponer una penalización alcohólica obligatoria.

---

14. EDITOR DE CARTAS

Implementa una sección accesible desde la navegación principal.

Debe permitir:

- Listar las cartas.
- Filtrar por categoría.
- Buscar por texto o ID.
- Agregar una carta.
- Editar una carta.
- Duplicar una carta con un ID nuevo.
- Desactivar o activar una carta.
- Eliminar una carta con confirmación.
- Ver una vista previa.
- Importar cartas JSON.
- Exportar cartas JSON.

Campos del formulario

- Categoría.
- Título.
- Descripción.
- Icono.
- Nivel de picante.
- Modo de asignación.
- Cantidad de participantes.
- Tipo de penalización.
- Cantidad de unidades.
- Manual asociado.
- Estado activo o inactivo.

El ID se genera automáticamente y no debe duplicarse.

Al guardar una carta, validar todos los campos antes de modificar el catálogo.

El formulario debe mostrar mensajes comprensibles cuando exista un error.

Al cancelar una edición, no debe modificar el contenido original.

---

15. MANUAL DE MINIJUEGOS

Los manuales deben estar integrados en la aplicación y ser accesibles desde un botón flotante circular con el símbolo `?`.

El botón debe estar fuera de la superficie de la carta, en una esquina de la pantalla, sin tapar el contenido ni los controles principales.

El manual se abre en una ventana modal o panel independiente.

Ejemplo de estructura:

```
{
  id: "MANUAL_SEMANA_INGLESA",
  title: "Semana inglesa",
  summary: "Resumen del minijuego.",
  instructions: [
    "Leer las reglas completas.",
    "Preparar a los participantes.",
    "Comenzar cuando todos estén listos."
  ],
  notes: "Reglas adicionales."
}
```

Estos textos son solo ejemplos de estructura. El creador incorporará las reglas reales.

Reglas:

- Cada minijuego puede tener un `manualId`.
- Si no existe un manual asociado, el botón puede mostrar «Sin manual disponible».
- Abrir y cerrar el manual no debe cambiar el turno.
- No se debe extraer otra carta al cerrar el manual.
- Las reglas no necesitan ejecutarse automáticamente.
- No implementar temporizadores ni puntuación en esta versión.
- Los manuales deben ser consultables desde una sección independiente.

Incluye un manual de ejemplo para «Semana inglesa», claramente identificado como provisional y editable.

---

16. ALMACENAMIENTO LOCAL

Utiliza `localStorage` para guardar:

- Cartas personalizadas.
- Catálogo de bebidas.
- Manuales.
- Participantes.
- Preferencias de asignación.
- Estado de los mazos.
- Carta actual.
- Jugador actual.
- Configuración visual.
- Estado de la partida.
- Historial mínimo de resultados.

Utiliza claves con versión, por ejemplo:

- `gdpw.cards.v1`
- `gdpw.manuals.v1`
- `gdpw.participants.v1`
- `gdpw.drinks.v1`
- `gdpw.gameState.v1`
- `gdpw.settings.v1`

El sistema debe:

1. Cargar datos al iniciar.
2. Validar los datos recuperados.
3. Guardar después de cada modificación importante.
4. Recuperar la partida tras una recarga.
5. Gestionar errores de lectura o escritura.
6. Ofrecer exportación de respaldo.
7. Evitar sobrescribir datos válidos con un archivo importado inválido.
8. Versionar los datos para permitir futuras migraciones.

No afirmes que `localStorage` garantiza que los datos nunca se perderán. Explica en la interfaz de configuración que los datos pertenecen al navegador y que conviene exportar respaldos.

---

17. EJECUCIÓN EN PC Y ANDROID

El primer entorno de desarrollo será un PC con Visual Studio Code.

Ejecución inicial en PC

Prepara el proyecto para ejecutarse con un servidor HTTP local sencillo.

Se puede utilizar Live Server en Visual Studio Code durante el desarrollo.

No abras el proyecto exclusivamente mediante `file://`, ya que los módulos JavaScript y las funciones de caché pueden comportarse de manera distinta.

Prueba desde Android

El usuario debe poder:

1. Conectar el PC y el teléfono Android a la misma red Wi-Fi.
2. Iniciar el servidor local.
3. Obtener la IP privada del PC.
4. Abrir en el navegador del teléfono una dirección del tipo `http://IP-LOCAL:PUERTO`.
5. Utilizar el juego desde el teléfono.

Documenta estos pasos en `README.md`.

El servidor debe escuchar en una interfaz de red accesible desde el teléfono, no solamente en `localhost`, cuando se requiera acceso por Wi-Fi. Explica que el firewall del PC puede necesitar una regla de acceso en la red privada.

Futura ejecución directamente en Android

Organiza el código para permitir que el proyecto se copie al teléfono y se ejecute mediante un servidor HTTP local en Android.

No asumas que abrir el HTML directamente desde un administrador de archivos garantiza el funcionamiento de todos los módulos JavaScript o de una PWA.

Incluye en el README una sección titulada «Ejecutar directamente en Android», con los requisitos y pasos para la futura implementación.

No es necesario crear una APK en esta primera versión.

Uso sin conexión

No dependas de servidores externos para las funciones esenciales.

Incluye localmente los archivos JavaScript, CSS y los iconos necesarios.

La futura versión sin conexión podrá utilizar un service worker y una PWA, pero no afirmes que esa función está terminada hasta haberla implementado y probado.

---

18. ESTRUCTURA DE ARCHIVOS

Organiza el proyecto de esta manera:

```
get-drunk-party-web/
├── index.html
├── README.md
├── manifest.json
├── css/
│   └── styles.css
├── js/
│   ├── app.js
│   ├── config.js
│   ├── cards.js
│   ├── participants.js
│   ├── dice.js
│   ├── decks.js
│   ├── game.js
│   ├── editor.js
│   ├── manuals.js
│   ├── storage.js
│   ├── drinks.js
│   ├── icons.js
│   └── utils.js
├── data/
│   ├── initial-cards.js
│   └── initial-manuals.js
├── assets/
│   └── icons/
└── tests/
    ├── dice.test.js
    ├── decks.test.js
    ├── participants.test.js
    └── cards.test.js
```

Puedes ajustar esta estructura si hay una razón técnica clara, pero conserva la separación entre datos, interfaz, lógica de juego y almacenamiento.

No crees archivos vacíos que aparenten implementar funciones que realmente no existen.

---

19. CATÁLOGO INICIAL: 50 CARTAS ORIGINALES

Implementa las siguientes 50 cartas de prueba.

Todas son contenido provisional, original y editable. No copies textos de las cartas comerciales de Open Party ni de Get Drunk.

Usa IDs únicos, categoría correcta, nivel de picante de 0 a 5, modo de asignación y penalización configurable.

Los valores de unidades de penalización son valores de prueba para la interfaz, no recomendaciones de consumo. Incluye siempre una alternativa sin alcohol.

A. VERDAD — 10 cartas

ID	Título	Descripción	Fuegos	Asignación	Unidades de prueba
VERDAD_001	Primera impresión	¿Qué impresión tuviste de la persona a tu derecha cuando la conociste?	1	CURRENT_PLAYER	1
VERDAD_002	Confesión divertida	Cuenta una situación vergonzosa que hoy te dé risa.	1	CURRENT_PLAYER	1
VERDAD_003	Secreto inocente	Comparte un dato curioso que casi nadie del grupo conozca.	1	CURRENT_PLAYER	1
VERDAD_004	Cita ideal	Describe tu cita ideal en tres frases.	2	CURRENT_PLAYER	1

VERDAD_005	Atracción	¿Qué cualidad te parece más atractiva en otra persona?	2	CURRENT_PLAYER	1
VERDAD_006	Momento incómodo	Cuenta una vez que intentaste impresionar a alguien y salió mal.	2	CURRENT_PLAYER	1
VERDAD_007	Pregunta del grupo	Deja que el grupo te haga una pregunta que puedas responder voluntariamente.	2	CURRENT_PLAYER	1
VERDAD_008	Primera cita	¿Qué detalle puede hacer que pierdas el interés en una cita?	3	CURRENT_PLAYER	1
VERDAD_009	Romance ficticio	Elige dos personajes ficticios que formarían una pareja inesperada y explica por qué.	2	CURRENT_PLAYER	1
VERDAD_010	Pregunta atrevida	Cuenta una anécdota romántica que estés dispuesto a compartir.	3	CURRENT_PLAYER	1

B. DESAFÍO — 10 cartas

ID	Título	Descripción	Fuegos	Asignación	Unidades de prueba
DESAFIO_001	Imitación	Imita durante 15 segundos a un personaje conocido para que el grupo lo adivine.	1	CURRENT_PLAYER	1

DESAFIO_002	Baile improvisado	Baila durante 20 segundos sin música y deja que el grupo elija el estilo.	1	CURRENT_PLAYER	1
DESAFIO_003	Elogio sincero	Dile un cumplido sincero a la persona de tu derecha.	1	RIGHT_PLAYER	1
DESAFIO_004	Voz de locutor	Presenta al siguiente jugador como si fuera una estrella mundial.	1	CURRENT_PLAYER	1
DESAFIO_005	Mirada seria	Mantén una expresión seria durante 20 segundos mientras otra persona intenta hacerte reír.	2	PLAYER_PAIR	1
DESAFIO_006	Escena romántica	Representa una escena romántica ficticia con otro participante, solo si ambos aceptan.	3	PLAYER_PAIR	1
DESAFIO_007	Declaración absurda	Declárale tu amor a un objeto de la habitación durante 15 segundos.	1	CURRENT_PLAYER	1
DESAFIO_008	Pasarela	Haz una pasarela de moda improvisada por la habitación.	2	CURRENT_PLAYER	1

DESAFIO_009	Dueto	Canta un fragmento de una canción junto a otra persona que acepte participar.	2	PLAYER_PAIR	1
DESAFIO_010	Reto de grupo	Propón una pose grupal y consigue que todos los participantes que quieran se unan.	2	ALL_PLAYERS	1

Las cartas de contacto físico que se incorporen posteriormente deberán poder rechazarse y tener una alternativa configurable. No asumas consentimiento por la mera asignación aleatoria.

C. MINIJUEGO — 10 cartas

ID	Título	Descripción	Fuegos	Asignación	Manual
MINIJUEGO_001	Semana inglesa	Jueguen a Semana inglesa siguiendo el manual configurado por el creador.	3	ALL_PLAYERS	MANUAL_SEMANA_INGLESA
MINIJUEGO_002	Mímica relámpago	Una persona representa una palabra y el grupo intenta adivinarla.	1	ALL_PLAYERS	MANUAL_MIMICA

MINIJUEGO_003	Palabras encadenadas	Cada persona dice una palabra relacionada con la anterior, sin repetir.	1	ALL_PLAYERS	MANUAL_PALABRAS
MINIJUEGO_004	Dos verdades ficticias	Cada jugador inventa dos afirmaciones y el grupo elige cuál parece más creíble.	2	ALL_PLAYERS	MANUAL_DOS_VERDADES
MINIJUEGO_005	Duelo de canciones	Dos personas se turnan para cantar fragmentos de canciones relacionadas con un tema elegido.	2	PLAYER_PAIR	MANUAL_DUELO_CANCIONES
MINIJUEGO_006	Historia colectiva	El grupo construye una historia añadiendo una frase por turno.	1	ALL_PLAYERS	MANUAL_HISTORIA

MINIJUEGO_007	Adivina el personaje	Un participante piensa en un personaje y los demás hacen preguntas de sí o no.	1	ALL_PLAYERS	MANUAL_PERSONAJE
MINIJUEGO_008	Categorías	El grupo elige una categoría y cada jugador nombra un elemento sin repetir.	1	ALL_PLAYERS	MANUAL_CATEGORIAS
MINIJUEGO_009	Escena improvisada	Dos jugadores interpretan una escena absurda sugerida por el grupo.	2	PLAYER_PAIR	MANUAL_IMPROVISACION
MINIJUEGO_010	Doblaje improvisado	Dos participantes inventan diálogos para una escena imaginaria.	2	PLAYER_PAIR	MANUAL_DOBLAJE

Los manuales serán editables. Los textos de las cartas no deben contener todas las reglas: la descripción es breve y el botón de ayuda abre el manual completo.

No inventes reglas tradicionales de Semana inglesa si no se han proporcionado. El manual inicial de ese juego debe indicar que las reglas completas están pendientes de ser introducidas por el creador.

D. PREMIO — 10 cartas

ID	Título	Descripción	Fuegos	Asignación	Unidades de prueba
PREMIO_001	Salvoconducto	Guarda esta carta para pasar una carta futura sin penalización.	1	CURRENT_PLAYER	0
PREMIO_002	Elegir canción	Elige la siguiente canción del grupo.	1	CURRENT_PLAYER	0
PREMIO_003	Cambia de turno	Puedes intercambiar tu siguiente turno con una persona que acepte.	1	PLAYER_PAIR	0
PREMIO_004	Pregunta libre	Elige a alguien que quiera responder una pregunta que tú inventes.	2	PLAYER_PAIR	0
PREMIO_005	Poder de veto	Guarda el derecho a vetar una carta futura una sola vez.	2	CURRENT_PLAYER	0
PREMIO_006	Elige categoría	Puedes solicitar una categoría distinta en tu siguiente turno, si hay cartas disponibles.	2	CURRENT_PLAYER	0
PREMIO_007	Turno protegido	Tu siguiente carta puede pasarse sin penalización.	1	CURRENT_PLAYER	0

PREMIO_008	Reto compartido	Invita a otra persona a completar contigo una actividad que ambos acepten.	2	PLAYER_PAIR	0
PREMIO_009	Director de escena	Elige una escena ficticia que dos voluntarios representarán.	2	ALL_PLAYERS	0
PREMIO_010	Comodín	Guarda esta carta para cambiar una carta futura por otra de la misma categoría.	2	CURRENT_PLAYER	0

Las cartas de premio deben tener un efecto explícito. Implementa solo los efectos sencillos definidos aquí. Si un premio necesita guardarse para el futuro, registra ese derecho en el estado de la partida y evita que pueda utilizarse más veces de las permitidas.

E. CASTIGO — 10 cartas

ID	Título	Descripción	Fuegos	Asignación	Unidades de prueba
CASTIGO_001	Rima improvisada	Inventa una rima corta con el nombre de otro participante.	1	CURRENT_PLAYER	1
CASTIGO_002	Acento inventado	Habla con un acento ficticio durante tu siguiente intervención.	1	CURRENT_PLAYER	1
CASTIGO_003	Estatua	Mantén una pose divertida mientras el grupo cuenta hasta diez.	1	CURRENT_PLAYER	1

CASTIGO_004	Elogios	Di algo positivo sobre tres personas del grupo.	1	CURRENT_PLAYER	1
CASTIGO_005	Anuncio comercial	Vende un objeto de la habitación como si fuera un producto revolucionario.	2	CURRENT_PLAYER	1
CASTIGO_006	Presentador	Presenta al siguiente jugador como si entrara a un programa de televisión.	1	CURRENT_PLAYER	1
CASTIGO_007	Canción inventada	Inventa una canción de dos líneas sobre la fiesta.	2	CURRENT_PLAYER	1
CASTIGO_008	Historia dramática	Cuenta una historia cotidiana como si fuera una tragedia épica.	2	CURRENT_PLAYER	1
CASTIGO_009	Cambio de personaje	Interpreta durante tu próxima intervención a un personaje inventado.	2	CURRENT_PLAYER	1
CASTIGO_010	Aplauso final	Dirige una ovación exagerada para todo el grupo.	1	ALL_PLAYERS	1

Los valores de penalización son datos editables de prueba. La aplicación no debe imponer que el participante consuma alcohol para resolver la carta. Debe permitir elegir una alternativa sin alcohol o pasar la actividad.

Configuración inicial de las 50 cartas

Cada carta debe tener un ID único, categoría, título, descripción, icono por defecto, nivel de picante, modo de asignación, configuración de penalización y estado activo.

No generes las 50 cartas a partir de una única descripción genérica. Registra cada carta de forma individual para que pueda editarse sin afectar las demás.

Los IDs de manual indicados en las cartas de minijuego deben existir en el catálogo de manuales, aunque algunos contengan inicialmente un aviso de «Reglas pendientes de completar».

---

20. PRUEBAS Y CRITERIOS DE ACEPTACIÓN

La aplicación se considerará funcional cuando se cumplan estas condiciones.

Dado

- Produce seis resultados posibles.
- Cada cara tiene la misma probabilidad teórica.
- SHOT no extrae ninguna carta.
- La animación muestra el resultado que realmente se generó.
- No se permiten tiradas simultáneas.

Cartas

- Cada categoría selecciona únicamente sus cartas activas.
- No se repite una carta durante el ciclo actual.
- El nuevo ciclo se baraja aleatoriamente.
- La primera carta del ciclo nuevo no coincide con la última del anterior cuando hay alternativas.
- Las categorías agotadas se gestionan con un aviso claro.

Participantes

- Se pueden registrar, editar y eliminar participantes.
- El orden de turnos se mantiene.
- El turno avanza una sola vez por resolución.
- La asignación aleatoria no elige participantes duplicados cuando se requieren personas distintas.
- Las restricciones configuradas se respetan.
- Si no existe una asignación válida, el juego ofrece una alternativa.

Editor

- Se pueden crear, editar, duplicar y eliminar cartas.
- Los IDs son únicos.
- Se validan los campos obligatorios.
- Los cambios se guardan localmente.
- Se pueden importar y exportar datos.
- Los archivos JSON inválidos se rechazan sin borrar los datos existentes.

Manuales

- El botón circular ? aparece fuera de la carta.
- Se abre el manual correcto.
- Cerrar el manual no cambia la carta ni el turno.
- Un manual pendiente se identifica claramente.

Persistencia

- Los cambios sobreviven a una recarga normal.
- El estado del mazo se recupera correctamente.
- Una operación incompleta no provoca una segunda tirada.
- Los errores de almacenamiento muestran un mensaje útil.

Android

- No existe desplazamiento horizontal accidental.
- Los botones son cómodos para tocar.
- El texto de las cartas se adapta al ancho.
- El servidor local permite abrir la web desde el teléfono.
- El proyecto no necesita conectarse a servicios remotos para las funciones esenciales.

---

21. PRUEBAS AUTOMATIZADAS

Crea pruebas para:

1. Distribución y validación de resultados del dado.
2. Bloqueo de lanzamientos simultáneos.
3. Barajado de mazos.
4. Prevención de repeticiones.
5. Restricción de la primera carta entre ciclos.
6. Avance circular de participantes.
7. Asignación de parejas distintas.
8. Manejo de categorías agotadas.

9. Validación de cartas.
10. Importación de JSON inválido.
11. Persistencia y recuperación de la partida.
12. Prevención de dobles resoluciones de turno.

No afirmes que la distribución es estadísticamente perfecta después de una prueba corta. La prueba debe comprobar que el algoritmo puede producir las seis caras y que la selección se realiza mediante una distribución uniforme del generador empleado.

Si incorporas un framework de pruebas, mantenlo como dependencia de desarrollo opcional y documenta cómo ejecutarlo. La aplicación de producción debe seguir funcionando con HTML, CSS y JavaScript puro.

---

22. DOCUMENTACIÓN

Crea un `README.md` que explique:

- Qué es el proyecto.
- Estructura de archivos.
- Cómo ejecutar en PC.
- Cómo acceder desde Android mediante Wi-Fi.
- Cómo agregar cartas editando los datos JavaScript.
- Cómo agregar cartas desde el editor.
- Cómo cambiar los colores de las categorías.
- Cómo agregar iconos locales.
- Cómo crear manuales de minijuegos.
- Cómo modificar el catálogo de bebidas.
- Cómo importar y exportar datos.
- Cómo realizar copias de seguridad.
- Qué limitaciones tiene el almacenamiento local.
- Cómo preparar una futura ejecución directamente en Android.

Incluye ejemplos concretos de una carta, un manual y un participante.

---

23. ORDEN DE IMPLEMENTACIÓN

Implementa el proyecto en este orden:

Fase 1. Estructura HTML y diseño responsive.

Fase 2. Participantes y orden de turnos.

Fase 3. Catálogo de cartas y categorías.

Fase 4. Dado de seis caras y animaciones.

Fase 5. Mazos sin repetición.

Fase 6. Asignación de participantes y resolución de cartas.

Fase 7. Editor de cartas.

Fase 8. Manuales y botón de ayuda.

Fase 9. Catálogo de bebidas y reglas de penalización.

Fase 10. Persistencia, importación y exportación.

Fase 11. Pruebas y corrección de errores.

Fase 12. Documentación y pruebas desde Android.

No dejes botones decorativos sin funcionalidad. Si un control no está implementado, no lo presentes como si ya funcionara.

---

24. ENTREGA FINAL

Al terminar, entrega:

1. El proyecto completo con todos los archivos.
2. Las 50 cartas iniciales.
3. Los manuales de ejemplo.
4. El editor de cartas.
5. El dado y las animaciones.
6. El módulo de participantes.
7. El sistema de mazos.
8. El almacenamiento local.
9. El catálogo configurable de bebidas.
10. El archivo `README.md`.
11. Las pruebas automatizadas y sus instrucciones.
12. Un resumen de las funciones verificadas.
13. Una lista de cualquier función pendiente o limitación encontrada.

No declares una prueba superada si no la ejecutaste. No sustituyas funcionalidades por comentarios TODO sin advertirlo.

Objetivo final: una aplicación de fiesta visualmente atractiva, funcional, fácil de ampliar y capaz de ejecutarse localmente desde un PC y utilizarse desde un teléfono Android sin depender de una plataforma en la nube.

Comienza inspeccionando el entorno y después construye el proyecto. Prioriza el funcionamiento real, la mantenibilidad y la experiencia móvil por encima de los efectos visuales innecesarios.